

Library Management System

Project Documentation

1. Introduction

This project is a console-based Library Management System built using C#, .NET, Entity Framework Core, and PostgreSQL. The main goal is to manage day-to-day library operations in a clean, structured way.

The system handles:

- Library Members
- Books and Book Copies
- Borrowing and Return Operations
- Fine Management
- Reports

The architecture follows enterprise-level backend principles:

- Layered Architecture
- Repository Pattern and Generic Repository Pattern
- Dependency Injection
- Separation of Concerns
- Custom Exception Handling

2. Technologies Used

- C# — Backend programming language
- .NET — Application framework
- PostgreSQL — Relational database
- Entity Framework Core — ORM for database interaction
- LINQ — Querying the database
- Dependency Injection — Managing services and dependencies
- EF Core Scaffold — Database First code generation

3. Database Design

The project uses a relational database. Below are all the tables with their columns and purpose.

MembershipTypes

Stores different membership plans available in the library.

- MembershipTypeId — SERIAL, Primary Key
- TypeName — VARCHAR, name of the plan

- MaxBorrowLimit — INT, how many books a member can borrow
- MaxBorrowDays — INT, maximum days a book can be kept

Members

Stores details of all registered library members.

- MemberId — SERIAL, Primary Key
- Name — VARCHAR, full name of the member
- Email — VARCHAR, unique email address
- Phone — VARCHAR, contact number
- Address — TEXT, residential address
- IsActive — BOOLEAN, whether the account is active
- MembershipTypeId — FK referencing MembershipTypes
- CreatedAt — TIMESTAMP, when the record was created

Categories

Stores book genres and categories.

- CategoryId — SERIAL, Primary Key
- CategoryName — VARCHAR, name of the category

Books

Stores all books available in the library collection.

- BookId — SERIAL, Primary Key
- Title — VARCHAR, book title
- Author — VARCHAR, author name
- ISBN — VARCHAR, unique ISBN number
- PublishedYear — INT, year of publication
- CategoryId — FK referencing Categories

BookCopies

Tracks physical copies of each book.

- CopyId — SERIAL, Primary Key
- BookId — FK referencing Books
- IsAvailable — BOOLEAN, whether the copy can be borrowed
- IsDamaged — BOOLEAN, whether the copy is damaged

Borrowings

Records every borrow and return transaction.

- BorrowingId — SERIAL, Primary Key
- MemberId — FK referencing Members
- CopyId — FK referencing BookCopies
- BorrowDate — TIMESTAMP, when the book was borrowed
- DueDate — TIMESTAMP, when it should be returned

- ReturnDate — TIMESTAMP, when it was actually returned (null if not yet)
- IsReturned — BOOLEAN, return status
- FineAmount — DECIMAL, fine charged for late return

4. Database Relationships

All relationships in this project are One-to-Many:

- MembershipTypes -> Members (one type, many members)
- Categories -> Books (one category, many books)
- Books -> BookCopies (one book, many physical copies)
- Members -> Borrowings (one member, many borrow records)
- BookCopies -> Borrowings (one copy, many borrow records)

5. ER Diagram (Text)



6. Database First Approach

This project uses the Database First approach. The database schema is created first in PostgreSQL, and then EF Core scaffolds the entity classes automatically.

Scaffold command used:

```

dotnet ef dbcontext scaffold
"Host=localhost;Port=5432;Database=librarydb;Username=postgres;Password=password"
Npgsql.EntityFrameworkCore.PostgreSQL --project LMS.Context --startup-project
LMS.FE --output-dir Entities --context LibraryDbContext --context-dir
DbContextFolder --force
  
```

This generates:

- Entity classes for each table
- DbContext with OnModelCreating configuration
- Navigation properties for all relationships

7. Project Architecture

The solution is split into 7 projects:

- LMS.FE — Frontend layer, console UI and menus
- LMS.BL — Business logic, services and validations
- LMS.DAL — Data access layer, repositories and queries
- LMS.Interfaces — All repository and service interfaces
- LMS.Models — Entity classes and enums
- LMS.Context — LibraryDbContext and EF Core config
- LMS.Exceptions — Custom exception classes

8. Layer Details

LMS.FE

Handles everything the user sees and interacts with.

- Program.cs — entry point
- Menus — MainMenu, AdminMenu, CustomerMenu
- Helpers — MemberHelper, BookHelper, BorrowingHelper, ReportHelper, InputHelper

LMS.BL

Contains all business rules and logic.

- MemberService
- BookService
- BorrowingService
- ReturnService
- FineService
- ReportService

LMS.DAL

Talks directly to the database.

- GenericRepository
- MemberRepository
- BookRepository
- BorrowingRepository
- FineRepository
- ReportRepository

LMS.Interfaces

Defines contracts for repositories and services. Keeps layers loosely coupled.

LMS.Exceptions

Custom exceptions for specific error scenarios.

9. Generic Repository Pattern

A generic repository was implemented to avoid writing the same CRUD code for every entity.

Interface: `IGenericRepository<T>`

Methods:

- `Add()`
- `Update()`
- `Delete()`
- `GetById()`
- `GetAll()`

This reduces duplicate code and makes it easy to maintain.

10. Dependency Injection

DI is set up using `Microsoft.Extensions.DependencyInjection` and configured in `ServiceConfiguration.cs`.

Why we use it:

- Keeps layers loosely coupled
- Makes it easier to swap implementations
- Dependencies are resolved automatically
- Easier to write unit tests

11. Features Implemented

Member Module

- Add Member
- View Members
- Search Member
- Deactivate Member

Book Module

- Add Book
- View Books
- Search Books
- Add Book Copies

Borrowing Module

- Borrow Book
- Return Book

- Fine Calculation
- Borrowing Validation

Reports Module

- Overdue Books Report
- Members With Pending Fine
- Most Borrowed Books
- Available Books Report
- Member Borrowing History

12. Custom Exceptions

Custom exceptions were created for each specific error case instead of using generic exceptions.

- MemberNotFoundException
- DuplicateEmailException
- BookUnavailableException
- BorrowLimitExceededException
- AlreadyReturnedException
- FineLimitExceededException

This makes error handling cleaner and easier to debug.

13. Role-Based Menu System

Two roles are supported: Admin and Customer.

Admin

- Manage Members
- Manage Books
- View Reports

Customer

- Search Books
- Borrow Books
- Return Books
- View Fine
- View Borrowing History

14. Future Enhancements

- ASP.NET Core Web API

- JWT Authentication
- Async Programming
- DTOs and AutoMapper
- Validation Framework
- Unit Testing with xUnit and Moq
- Logging with Serilog
- Docker Deployment

15. Conclusion

This project covers all the core concepts of building a backend application using .NET — layered architecture, EF Core with Database First, the generic repository pattern, DI, custom exceptions, and a role-based console UI.

It's a solid foundation for moving on to more advanced topics like Web APIs, async programming, and containerised deployments.